159.333 Computer Science Project

Bargain Agent

An agentic AI system that finds products and monitors bargain prices across New Zealand retailers

Final Project Report

Massey University · Semester 2, 2026 · October 2026

Table 1. Project team

Name	Student ID	Role
Fengwei Zhang	24009491	Project Lead
Gang Zhao	24009365	Data Engineer
Tianshuo Gao	24009350	Interface Developer
Xiao Zhang	24009323	Agent Developer and Test Engineer
Wenhan Zhang	24009400	Agent Developer and Test Engineer

Supervisor: [supervisor name]

Contents

1. Introduction 2
1.1 Purpose of the project 2
1.2 Who will use it 2
1.3 Goals 2
1.4 Team and module ownership 2
1.5 Use of generative AI 3
2. Background 4
3. Project evolution and key decisions 5
3.1 The original approach: scraping each retailer 5
3.2 What went wrong 5
3.3 Decision: read Google Shopping through search APIs 5
3.4 Decision: a pipeline of single-purpose AI agents 5
3.5 Other decisions 6
4. System design 7
4.1 Architecture 7
4.2 Main flows 7
4.3 Technologies 7
5. Implementation 9
5.1 The eight AI agents 9
5.2 Connecting the agents with LangGraph 9
5.3 One validated entry point for the model 10
5.4 Data providers and price verification 10
5.5 Same-product matching 10
5.6 Monitoring and alerts 10
5.7 Web interface and email 11
5.8 Security and cost control 13
5.9 Packaging and demo mode 13
6. Testing and evaluation 14
6.1 Automated tests 14
6.2 Evaluation plan 14
6.3 Real price accuracy 14
6.4 Agents with the real model 15
6.5 End to end 15
6.6 Limits of the evaluation 15
7. Problems and how we solved them 16
8. User manual 17
8.1 Running the Windows program 17
8.2 Running from source 17
8.3 Finding a product and creating a monitor 17
8.4 Checking without keeping the window open 17
9. Conclusion 18
10. Future work 19
11. Personal learning 20
References 21
Source code 22

1. Introduction

1.1 Purpose of the project

Online prices in New Zealand change often and differ between shops. A person who wants a pair of headphones or an air fryer usually checks PB Tech, Noel Leeming, JB Hi-Fi or Mighty Ape by hand, again and again, and still misses the sale. Existing price alert tools usually require the user to find the exact product page first and fill in a form.

Bargain Agent lets a user describe what they want in their own words, in English or Chinese, for example *"noise cancelling headphones under NZD 400"*. A pipeline of AI agents turns the description into real products found in New Zealand shops, compares the same product across merchants, keeps watching the price, and sends an email when the price becomes a real deal. The project brings together three ideas from our proposal: autonomous shopping agents, natural-language interaction through a large language model (LLM), and scheduled monitoring with notifications.

1.2 Who will use it

The system is a personal tool for one user on their own Windows computer: a shopper in New Zealand who wants to buy a specific kind of product at a good price but does not want to check shops every day. It runs locally and is used through a web page in the browser. A command-line interface is also available for technical users and for scheduled checks.

1.3 Goals

The proposal set five goals. All five were delivered, and the way they were delivered changed during the project, as Section 3 explains.

- Autonomous, scheduled price monitoring across several New Zealand retailers.
- Natural-language interaction, so that monitoring tasks are created conversationally.
- A reliable notification system that alerts the user to price drops.
- An evaluation of price extraction, instruction understanding and alert triggering.
- A working Python prototype with clear documentation.

1.4 Team and module ownership

The system was split into four modules along the flow of data, and each member owned one part. The project lead owned the architecture, the integration code that connects the modules, the evaluation summary and the delivery. Table 2 lists the owners; the complete list of files per member is kept in `docs/TEAM_RESPONSIBILITIES.md` in the source code.

Table 2. Module ownership

Member	Owns
Fengwei Zhang	Architecture and key decisions; integration code (`pipeline.py`, `llm.py`, `config.py`); evaluation summary; packaging and delivery; documentation

Member	Owns
Gang Zhao	Data layer: search providers and page readers (`sources/`), price observation (`pricing.py`), SQLite storage (`storage.py`), real price collection
Tianshuo Gao	Interface: web UI (`static/index.html`, `webapp.py`), command-line task management (`cli.py`), email notifications (`notifier.py`)
Xiao Zhang	Find agents (Understand, Rewrite, Select, Describe) and same-product matching (`matching.py`, offer verifier)
Wenhan Zhang	Monitoring loop and deal decisions (`scheduler.py`), instruction parsing and price extraction agents, output validation (`schemas.py`)

[Optional, from the report guide: a short statement of which member wrote which section of this report.]

1.5 Use of generative AI

The team used AI coding assistants during development to help write and review code, tests and documentation, and to draft parts of this report. The team made the design decisions, checked the generated work, and verified the system through automated tests and real-data evaluation (Section 6). The agents inside Bargain Agent itself use the DeepSeek language model (DeepSeek, n.d.).

[Team: name the AI tools you used and adjust this statement to match the course policy.]

2. Background

Shopping agents are an old idea. ShopBot learned how to query online stores and extract prices, and compared offers across vendors automatically (Doorenbos et al., 1997). Maes et al. (1999) described how software agents could take over parts of buying and selling, such as finding products, comparing merchants and negotiating. More recently, Jorge et al. (2020) showed that web scraping can support price monitoring in a real retail sector, but scraping depends on the structure of each site.

Large language models make a new kind of agent possible. ReAct showed that a model can interleave reasoning with actions such as searching (Yao et al., 2023), and Wang et al. (2024) survey the growing field of LLM-based autonomous agents, which plan, use tools and act on behalf of a user. The same models can also produce fluent text that is not true, a problem known as hallucination (Ji et al., 2023). For a shopping agent, an invented price or product would be worse than no answer at all. This shaped the most important design rule of our system: **agents choose, code decides** (Section 3.4).

Our project combines these threads: the comparison-shopping idea of ShopBot, LLM agents for understanding and choosing, and strict code-level checks so that every price shown and every email sent can be traced to real data.

3. Project evolution and key decisions

The final system is quite different from the proposal. This section explains what changed and why, because these decisions explain most of the design.

3.1 The original approach: scraping each retailer

The proposal planned a web scraping module built with Requests and BeautifulSoup for static pages and Selenium for pages rendered by JavaScript (Jorge et al., 2020). In the first weeks we built retailer adapters for PB Tech and JB Hi-Fi, and verified real prices from both. A monitoring task, created from a sentence by an LLM parser, could watch a product page and send an email when the price fell below a threshold.

3.2 What went wrong

When we tried to cover more shops, the approach broke down:

- Mighty Ape returned HTTP 403 for the whole site, served by an anti-bot service (DataDome). Noel Leeming and other detail pages were also blocked or rendered only in the browser.
- Every retailer needed its own parser, and a parser stopped working whenever the shop changed its page layout.
- Finding products from a description required searching every shop separately, which multiplied both problems.

We decided not to work around anti-bot protection. Bypassing it would be fragile and would not respect the shops' choices.

3.3 Decision: read Google Shopping through search APIs

We decided to read Google Shopping New Zealand through commercial search APIs instead of scraping shops, because one search returns structured listings, with title, merchant, price and a product identifier, for every merchant at once. We use **Serper** as the main provider and **SerpApi** as a backup and for product details (Serper, n.d.; SerpApi, n.d.). Both return the same Google product identifiers, so a monitor can later find the same listing again. We only read a shop's own page when the user pastes a specific link, and then through a chain of three page-reading services (Scrape.do, Firecrawl and Tavily), so one blocked service does not stop the feature (Firecrawl, n.d.).

We considered a general web search API (Brave Search) and an AI search API (Tavily), which we had integrated earlier for finding candidate links. They return web pages, not prices, so every result still had to be scraped. A shopping API removed that step.

3.4 Decision: a pipeline of single-purpose AI agents

We decided to build the "find" feature as a pipeline of small agents, each with one job, instead of one large prompt. Small agents are easier to test, validate and replace, and if one fails only that step falls back to plain code. We built the pipeline with LangGraph, because our flow is a graph with a

condition (go back and search again if nothing suitable was found) and LangGraph expresses that directly while keeping a shared state between steps (LangChain, n.d.).

Because of the hallucination risk discussed in Section 2, we set one rule for every module: **agents choose, code decides**. An agent may only pick listings by identifiers that our server assigned, so it cannot invent a product or a price. Every model output is validated. Budgets, deal thresholds, duplicate checks and whether an email may be sent are decided by plain code that we can test.

3.5 Other decisions

Table 3. Further design decisions and the reasons for them

Decision	Reason
Python 3.13 with few dependencies	The team knew Python; a small dependency list (Requests, LangGraph, python-dotenv, schedule) keeps installation and packaging simple.
A local web app on 127.0.0.1, not a cloud service	It is a personal tool for one user; API keys and data stay on the user's computer and no server is needed.
SQLite for storage	No database server to install; price history fits easily; schema versions are handled with simple migrations (Python Software Foundation, n.d.-a).
DeepSeek through an OpenAI-compatible API	Supports JSON output and is low cost; because the client is compatible, models can be changed in the settings file, even per agent (DeepSeek, n.d.).
Only trusted merchants trigger alerts	Unknown marketplace and overseas sellers are shown for comparison, but an email should only go out for a deal that is safe to buy.
Email only, no SMS	The proposal mentioned SMS. Email was enough for our users, and SMS needs a paid service, so we left it as future work.
A Windows package built with PyInstaller	Teammates and users can run the system without installing Python (PyInstaller Development Team, n.d.).

All decisions, with their dates and reasons, are recorded in `project_progress/DECISIONS.md` in the source code.

4. System design

4.1 Architecture

Figure 1 shows the architecture. The system has four layers, each owned by one team member: the interface the user sees, the AI agents, the monitoring loop, and the data layer that reads prices and stores them. The layers do not depend on each other's internal details. They meet through a small integration layer: the LangGraph pipeline, a single entry point for all model calls, and configuration. The interfaces between layers were written down first, in `docs/INTERFACES.md`, so that team members could work in parallel.

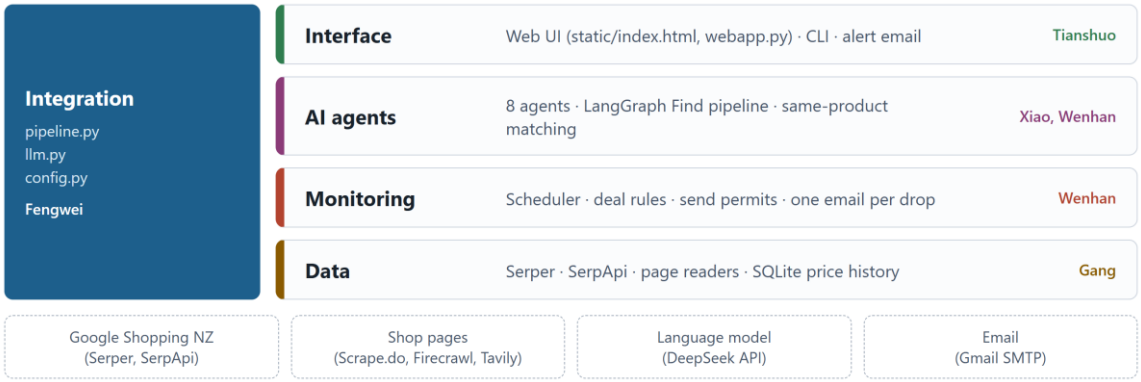


Figure 1. Architecture: four layers connected by an integration layer, and the external services the system uses.

4.2 Main flows

The system has two main flows (Figure 2). In **Find**, a description goes through the Understand, Rewrite, Select and Describe agents, with a plain-code Search step that calls the shopping API. If Select finds nothing suitable, the graph goes back to Rewrite for new search terms, within a fixed budget. In **Monitor**, the chosen product is matched at other merchants, stored as a task with up to five merchants, and checked on a schedule; a deal at a trusted merchant sends one email.



Figure 2. The Find pipeline (top) and the monitoring flow (bottom). Purple boxes are AI agents; all decisions about deals and emails are made in code.

4.3 Technologies

Table 4. Technologies used

Area	Technology	Used for
Language	Python 3.13	All application code and tests
Agent orchestration	LangGraph 1.2	The Find pipeline as a state graph with a conditional retry edge
Language model	DeepSeek (deepseek-flash), OpenAI-compatible API	Eight agents; JSON output mode
Shopping data	Serper (primary), SerpApi (backup, product details)	Google Shopping New Zealand listings and prices
Page reading	Scrape.do, Firecrawl, Tavily	Prices from a product page the user pastes
Storage	SQLite	Tasks, merchants, price history, sent alerts, API usage
Web interface	Python http.server, HTML, CSS, JavaScript	Local web app; price chart drawn as SVG
Email	smtplib with Gmail (app password)	Alert emails (Python Software Foundation, n.d.-b)
Testing	unittest, recorded API responses, fake model	362 offline tests and an evaluation runner
Packaging	PyInstaller	Windows program, no Python needed

The application has about 6,600 lines of Python in 32 files, a 1,000-line web page, eight agent prompts, and about 4,500 lines of tests.

5. Implementation

5.1 The eight AI agents

Table 5 lists the agents. Each agent is a small Python module with a prompt file, a validation function and a rule-based fallback.

Table 5. The eight AI agents

Agent	Job	Used in
Understand	Turns a description into a structured need: category, budget, currency, condition, brand	Find
Rewrite	Writes up to three search queries	Find
Select	Picks up to eight products from at most 30 listings, by listing ID only	Find
Describe	Writes a short summary of each pick, in English with a Chinese version	Find
Offer verifier	Checks whether another merchant sells the same product; it can only veto	Compare, monitor
Task parser	Turns a one-sentence instruction into a monitoring task, or asks for missing details	Add by sentence
Price extractor	Reads the price from the text of a shop page	Pasted links
Deal summary	Optional short summary at the top of an alert email (off by default)	Alert email

5.2 Connecting the agents with LangGraph

The Find pipeline is built in `build_graph()` in `pipeline.py`. Each step is a node; the edges fix the order, and a conditional edge after Select decides whether to retry. The decision function checks that Select ran successfully but chose nothing, and that there is still rewrite budget, search budget and time left (at most three queries and 120 seconds per search). Each node is wrapped so that it reports its name to the web page, which shows a progress bar. Several search queries run in parallel threads.

Code excerpt 1. pipeline.py, build_graph(): the Find pipeline as a LangGraph state graph.

```
graph = StateGraph(FindState)
graph.add_node("understand", staged("understand", understand_node))
graph.add_node("rewrite", staged("rewrite", rewrite_node))
graph.add_node("search", staged("search", search_node))
graph.add_node("select", staged("select", select_node))
graph.add_node("describe", staged("describe", describe_node))
graph.add_edge(START, "understand")
graph.add_edge("understand", "rewrite")
graph.add_edge("rewrite", "search")
graph.add_edge("search", "select")
graph.add_conditional_edges("select", after_select,
                            {"rewrite": "rewrite", "describe": "describe"})
graph.add_edge("describe", END)
```

A user can also refine a result in conversation, for example *"only trusted NZ shops"* or *"只要 Sony 的"*. The earlier listings are kept, at most two new searches are made, and the agents choose again.

5.3 One validated entry point for the model

All eight agents call the model through one function, `call_json()` in `llm.py`. It asks for JSON output, passes the parsed answer to the agent's own validation function, and retries once with the error fed back if the answer is invalid or was cut off. If it still fails, it raises an error and the agent uses its rule-based fallback, so a search still returns results and the page reports that some agents used fallbacks. Each agent can be given a different model in the settings file.

Code excerpt 2. llm.py, call_json() (signature shortened).

```
def call_json(prompt_name: str, payload: Any, *,
              validate=None, agent=None, retries=1,
              max_tokens=MAX_RESPONSE_TOKENS) -> Any:
    """Run one agent prompt and return its validated JSON output."""
```

A real problem shaped this function. The model we use reasons before it answers, and the reasoning counted against the response limit, so some answers were cut off and the Select agent fell back to plain ranking. Detecting a cut-off answer and raising the limit fixed it.

5.4 Data providers and price verification

All providers implement the same small interfaces in `sources/base.py` (shopping search, offer source, page extractor), so a provider can be replaced without changing the rest. A fallback wrapper uses SerpApi when Serper fails. Every provider call passes through a quota guard that counts calls per day in SQLite and stops at a configured limit.

Prices read from shop pages need extra care. When a page contains exactly one structured product offer (JSON-LD, as defined by Schema.org), we use that price directly (Schema.org, n.d.). Otherwise the price extractor agent reads the page text, and its answer is only accepted if the same price appears on the page. The prompt excludes prices before GST, instalment amounts and recommended retail prices, and treats a bare "$" on a .nz site as NZD.

5.5 Same-product matching

When the user chooses a product, `matching.py` looks for the same product at other merchants. Rules come first: brand and model words must match, the item must be new unless the user asked otherwise, and the price must be within 50% to 150% of the chosen product, which removes accessories and bundles. The offer verifier agent then checks the remaining candidates and can only reject them. One listing per merchant is kept, at most five merchants are watched, and trusted merchants are ticked by default (Figure 4).

5.6 Monitoring and alerts

Each scheduled check runs one shared Google Shopping search for the whole task, however many merchants it watches, and reads each merchant's current price by product identifier. If a merchant is missing from the results, at most two retry searches and two detail lookups are made. The scheduler then excludes offers the verifier marked as different, takes the cheapest offer from a trusted merchant, and decides in code whether it is a deal: the price is at or below the target, or it has dropped by a chosen percentage since the last alert.

A send permit in the database ensures that one price drop sends one email, even if two checks run at the same time, and task locks prevent duplicate checks. From the stored history the system also computes the 30-day low and average from the cheapest usable price of each check, and can compare delivered prices (item plus shipping) when shipping is known. Monitoring can run continuously in the web app, or once per run through a Windows scheduled task (`check-due`).

5.7 Web interface and email

The interface is a local web page served by Python's standard HTTP server (Figures 3 to 5). A search runs as a background job, and the page polls its progress so the user sees which agent is working. Results are cards with the price, the merchant, whether it is trusted, and the summary in English and Chinese. The detail view shows a price chart drawn as SVG, the latest price at each merchant, shipping, and the alert settings.

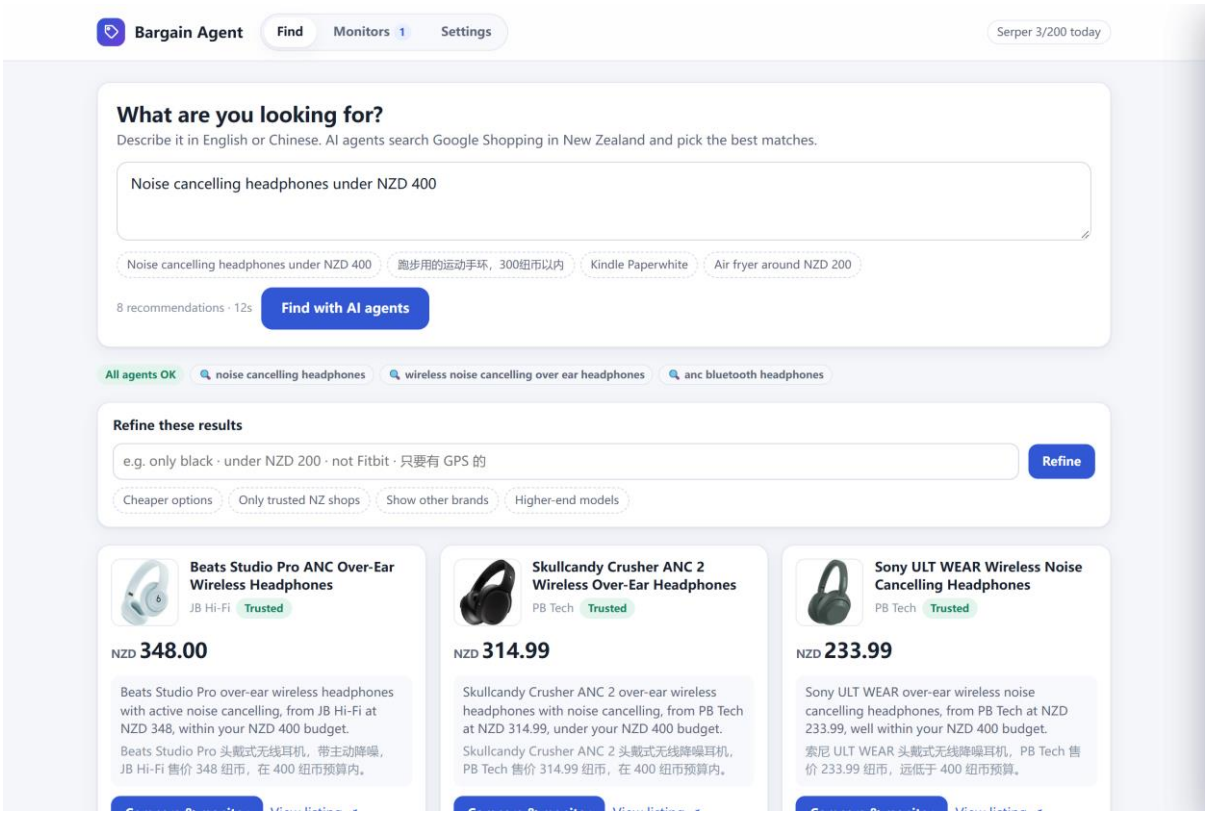


Figure 3. A real search for "noise cancelling headphones under NZD 400": the search terms written by the Rewrite agent, and product cards with bilingual summaries.

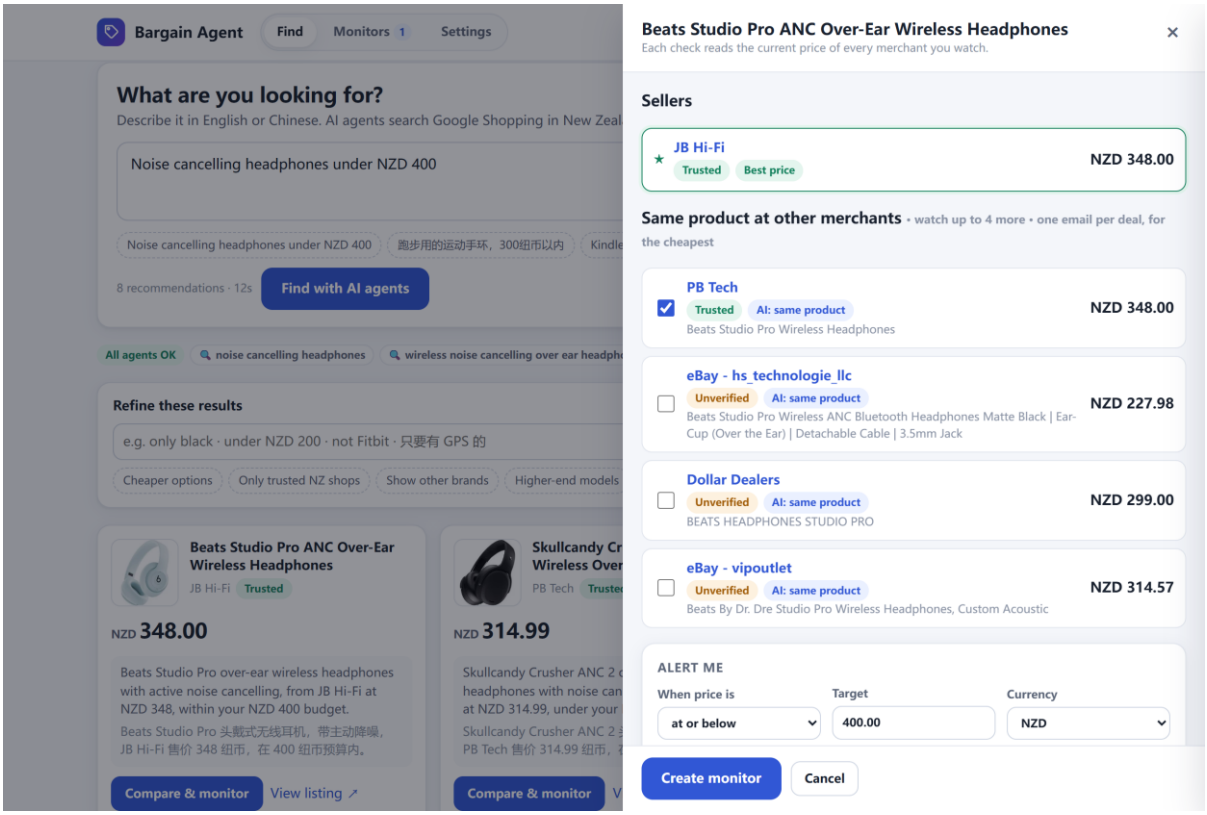


Figure 4. Compare and monitor: the same product at other merchants, checked by the offer verifier ("AI: same product"). Trusted merchants are ticked by default.

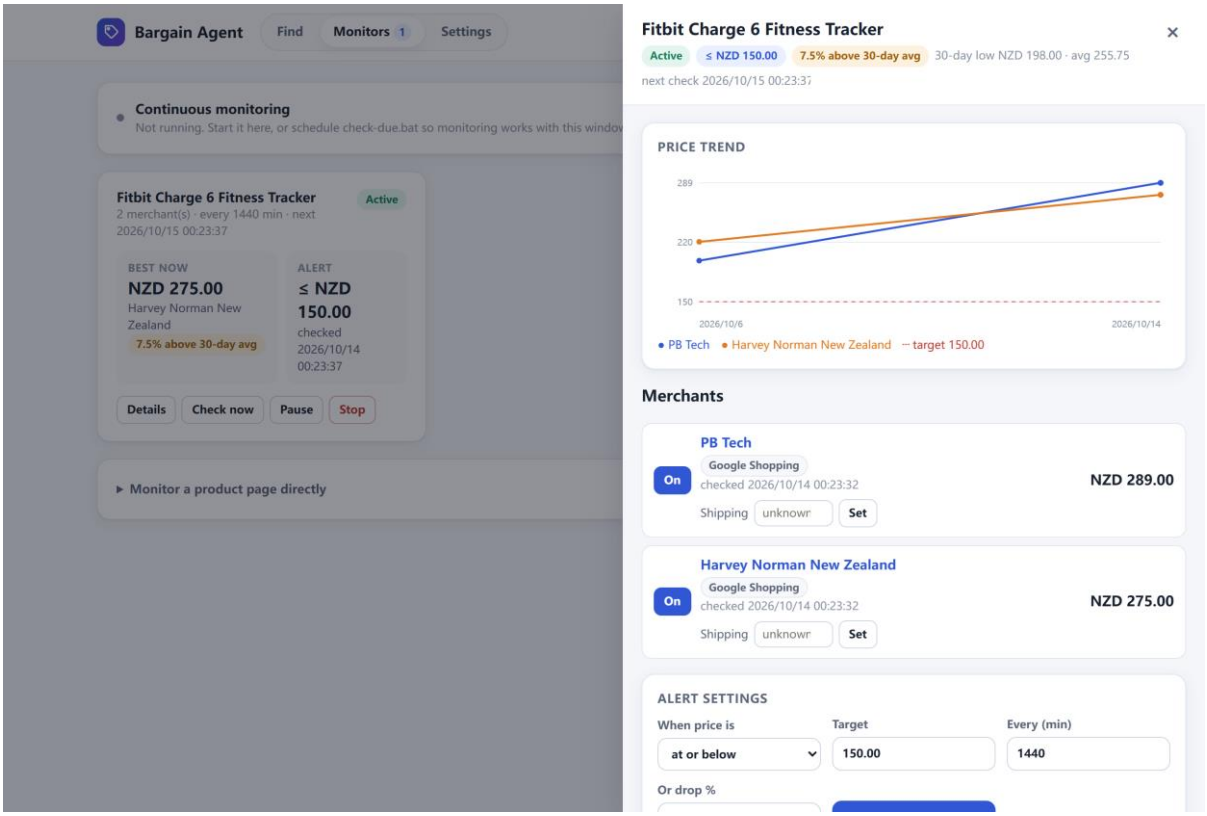


Figure 5. A real monitor for the Fitbit Charge 6 at two merchants, with its price trend from 6 to 14 October 2026 and the 30-day comparison. The owner's email address is hidden.

The alert email is built in `notifier.py` and sent through Gmail's SMTP server with an app password. Below is the text of a real alert sent by the system on 13 October 2026, after a test target of NZD 280 was set on the monitor in Figure 5. It also shows a useful property of the 30-day line: the price met the target, but the email tells the user it is not below the recent average.

```
Subject: [Bargain Agent] 降价提醒：Fitbit Charge 6 Fitness Tracker — NZD 275.00 @ Harvey
Norman New Zealand

降价提醒 / Price alert: Harvey Norman New Zealand 现价 NZD 275.00（目标 NZD 280.00）
A price match was found for your monitored item.
Product: Fitbit Charge 6 Fitness Tracker - Silver Aluminum Case with Porcelain Band ...
Current price: NZD 275.00
Your threshold: NZD 280.00 (lte)
Price history: Not below the 30-day average (low NZD 198.00, average NZD 255.75, 4 checks).
Retailer: Harvey Norman New Zealand
In stock: True
Shipping cost: unknown (price does not include verified shipping).
Other monitored sources (1):
  - PB Tech: NZD 289.00
```

5.8 Security and cost control

- The server listens only on 127.0.0.1. Every request that changes data needs a CSRF token, and the Origin and Host headers are checked (OWASP Foundation, n.d.). Request bodies are limited to 64 KB.
- API keys are kept in a local `.env` file that git ignores. The settings page can set the recipient and sending mailbox, but never shows saved values.
- Each provider has a daily call limit (by default 200 for Serper and 20 for SerpApi). One check uses one shared search, a task watches at most five merchants, and a search uses at most three queries.
- Checking a task or starting continuous monitoring asks for confirmation, because it can send a real email.

5.9 Packaging and demo mode

`build_package.py` builds a Windows program with PyInstaller. A user unzips it and double-clicks `BargainAgent.exe`, which starts the server and opens the browser. The settings file, database and logs live next to the program. A source package installs its own Python environment on first run. For presentations we added a demo mode with a separate database, in which earlier price points are simulated around real current prices and the page shows a clear "Demo data" banner; search and price checks stay live.

6. Testing and evaluation

6.1 Automated tests

The project has 362 automated tests that run offline in about two minutes. They cover every agent, the providers (using recorded real responses), matching, the scheduler, database migrations, the web API and the command line. Model answers are replaced by a fake client, and a guard test fails if any test tries to reach the real network, so the tests are free and repeatable. We followed a simple rule throughout: a change was complete only when all tests passed (Sommerville, 2016).

6.2 Evaluation plan

The proposal defined three accuracy measures: price extraction, instruction parsing and notification triggering. We measured all three with real data on 5 October 2026, using the DeepSeek model. Figure 6 summarises the results, and `docs/EVALUATION.md` gives the details.



Figure 6. Evaluation results on real data (5 October 2026).

6.3 Real price accuracy

For 17 common New Zealand products we took the first new, trusted-merchant listing from Google Shopping, opened the merchant's own page, read its price, and compared it with Google's price. Two queries had no suitable trusted listing. The merchants were PB Tech, Noel Leeming, Heathcotes, Mighty Ape and Harvey Norman.

Table 6. Real price accuracy

Run	Page reading	Compared	Exact match
First run	Firecrawl JSON extraction	15	9 (60%)
Re-read after fixes	JSON-LD first, else AI on page text	14	14 (100%)

We analysed every miss in the first run. The causes were specific and each led to a fix:

Table 7. Causes of wrong prices and their fixes

Cause	Example	Fix
Superscript cents read as one number	"$ 448 00" read as 44800	Use the page's JSON-LD price; an AI price must appear on the page

Cause	Example	Fix
Instalment taken as the price	Afterpay amount, a quarter of the price	Prompts exclude instalment amounts
Recommended retail price taken	RRP instead of the sale price	Prompts exclude was/RRP prices
Price before GST	"$172.17 +GST / $198.00 inc GST"	Prompts ask for the tax-inclusive price

6.4 Agents with the real model

Table 8. Real model evaluation

Track	Cases	Result
Price extraction from page text	13	13/13 exact; false-price rate 0
Instruction parsing	16	15/16 fully correct; 98.1% of fields
Offer price parsing (code)	26	100%
Alert triggering (real scheduler)	10 scenarios, 19 steps	Precision 1.0, recall 1.0, no unexpected email

The price extractor never reported a false price, including on a page that tried to inject instructions. The one instruction that was not fully parsed contained only a link and no product name; the model asked for clarification instead of guessing, which is the behaviour we want.

6.5 End to end

We also ran the whole system with real services: a real search for a Fitbit Charge 6, a monitor on two merchants (PB Tech and Harvey Norman, matched by the verifier), a real check that sent one email, and a second check that correctly sent no duplicate. The alert in Section 5.7 is a later real email from the same monitor.

6.6 Limits of the evaluation

The real price sample was 17 products, slightly below the 20 we planned, because each product costs paid API calls. We have not yet run the one-week continuous monitoring test from the proposal. Prices on shop pages can also change between the moment Google records them and the moment we read the page, so a small number of mismatches in a larger sample would not necessarily be errors.

7. Problems and how we solved them

Table 9. Main problems during the project

Problem	What we did
Retailers blocked our scraper (HTTP 403, anti-bot services)	Changed to shopping search APIs instead of bypassing protection (Section 3.3)
Wrong prices from shop pages	Analysed each miss; JSON-LD first, on-page check for AI prices, stricter prompts (Section 6.3)
Model answers cut off, agent fell back	Detected cut-off answers and raised the response limit for reasoning models
Google product links from the API showed "no details" when opened directly	Links now open a normal search for the product and merchant
The first real alert email was not noticed	Made the subject distinctive: "[Bargain Agent] 降价提醒 … @ merchant"
Saved search results appeared in the wrong mode (demo vs real)	Results are saved per database, identified by a random instance id
Teammates could not create monitors in the demo package	Demo mode works without an email address; errors also appear as pop-up messages
Installation failed in very long folder paths on Windows	The start script explains the problem and suggests a short path

8. User manual

8.1 Running the Windows program

1. Unzip `BargainAgent-windows.zip` to a normal folder such as the Desktop.
2. Double-click `BargainAgent.exe`. A console window opens and the browser shows http://127.0.0.1:8765/. Keep the window open while using the app.
3. On the Settings page, enter the email address that should receive alerts and the sending mailbox (for Gmail, an app password).

8.2 Running from source

Install Python 3.10 or newer, unzip the source package, and double-click `start.bat`. On first run it creates a Python environment and installs the requirements. Without keys it opens `.env` so that the language model and search keys can be entered.

8.3 Finding a product and creating a monitor

1. On the Find page, describe the product in English or Chinese and press *Find with AI agents*.
2. Optionally refine the results, for example *Only trusted NZ shops*.
3. Press *Compare & monitor* on a card, tick the merchants to watch, set the target price, and press *Create monitor*.
4. On the Monitors page, use *Check now* for an immediate check, or *Start monitoring* to check on schedule. *Details* shows the price chart and merchants.

8.4 Checking without keeping the window open

Register the scheduled check with Windows Task Scheduler, for example every 30 minutes:

```
schtasks /Create /SC MINUTE /MO 30 /TN "BargainAgent check-
due" /TR "<folder>\BargainAgent.exe --check-due"
```

Each task is still checked only at its own frequency. The computer must be on at check time.

9. Conclusion

Bargain Agent meets the goals of our proposal. A user can describe a product in their own words, see real products from New Zealand shops chosen by AI agents, compare the same product across merchants, and receive an email when the price meets their target. The most important lesson of the project was a decision rather than a feature: when scraping retailers failed, we changed our data source instead of fighting anti-bot protection, and that made the rest of the system possible.

The second lesson was about trust. By letting agents choose only from real listings and keeping every decision about money and emails in code, we could test the system and measure it: prices matched the shops' own pages in 14 of 14 cases after our fixes, the price agent never reported a false price, and alerts were triggered with precision and recall of 1.0. The main limitations are the size of the real sample, the dependence on what Google Shopping New Zealand lists, and the need for the computer to be on for monitoring.

10. Future work

- Run the one-week continuous monitoring test and extend the real price sample to 20 or more products.
- A cloud version, so that monitoring continues when the user's computer is off, with user accounts and shared searches for popular products.
- More alert channels: SMS (from the proposal), Telegram or desktop notifications.
- Let the user edit the list of trusted merchants in the interface.
- Support other countries by changing the Google Shopping locale and the trusted merchant list.
- Publish the code to a shared repository with continuous integration, so every change runs the tests automatically.

11. Personal learning

Each team member wrote their own paragraph.

Fengwei Zhang

[To be written by Fengwei Zhang: what you personally learned from the project, for example about the module you owned, a problem you solved, working in a team, or a technology you used for the first time. About 100–200 words.]

Gang Zhao

[To be written by Gang Zhao: what you personally learned from the project, for example about the module you owned, a problem you solved, working in a team, or a technology you used for the first time. About 100–200 words.]

Tianshuo Gao

[To be written by Tianshuo Gao: what you personally learned from the project, for example about the module you owned, a problem you solved, working in a team, or a technology you used for the first time. About 100–200 words.]

Xiao Zhang

[To be written by Xiao Zhang: what you personally learned from the project, for example about the module you owned, a problem you solved, working in a team, or a technology you used for the first time. About 100–200 words.]

Wenhan Zhang

[To be written by Wenhan Zhang: what you personally learned from the project, for example about the module you owned, a problem you solved, working in a team, or a technology you used for the first time. About 100–200 words.]

References

Doorenbos, R. B., Etzioni, O., & Weld, D. S. (1997). A scalable comparison-shopping agent for the World-Wide Web. In *Proceedings of the First International Conference on Autonomous Agents* (pp. 39–48). ACM. https://doi.org/10.1145/267658.267666

DeepSeek. (n.d.). *DeepSeek API documentation*. Retrieved October 18, 2026, from https://api-docs.deepseek.com/

Firecrawl. (n.d.). *Firecrawl documentation*. Retrieved October 18, 2026, from https://docs.firecrawl.dev/

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys, 55*(12), 1–38. https://doi.org/10.1145/3571730

Jorge, O., Pons, A., Rius, J., Vintró, C., Mateo, J., & Vilaplana, J. (2020). Increasing online shop revenues with web scraping: A case study for the wine sector. *British Food Journal, 122*(11), 3383–3401. https://doi.org/10.1108/BFJ-07-2019-0522

LangChain. (n.d.). *LangGraph documentation*. Retrieved October 18, 2026, from https://langchain-ai.github.io/langgraph/

Maes, P., Guttman, R. H., & Moukas, A. G. (1999). Agents that buy and sell. *Communications of the ACM, 42*(3), 81–91. https://doi.org/10.1145/295685.295716

OWASP Foundation. (n.d.). *Cross-site request forgery prevention cheat sheet*. Retrieved October 18, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

PyInstaller Development Team. (n.d.). *PyInstaller manual*. Retrieved October 18, 2026, from https://pyinstaller.org/en/stable/

Python Software Foundation. (n.d.-a). *sqlite3: DB-API 2.0 interface for SQLite databases*. Retrieved October 18, 2026, from https://docs.python.org/3/library/sqlite3.html

Python Software Foundation. (n.d.-b). *smtplib: SMTP protocol client*. Retrieved October 18, 2026, from https://docs.python.org/3/library/smtplib.html

Schema.org. (n.d.). *Offer*. Retrieved October 18, 2026, from https://schema.org/Offer

SerpApi. (n.d.). *Google Shopping API*. Retrieved October 18, 2026, from https://serpapi.com/google-shopping-api

Serper. (n.d.). *Serper: The world's fastest and cheapest Google Search API*. Retrieved October 18, 2026, from https://serper.dev/

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J.-R. (2024). A survey on large language model based autonomous

agents. *Frontiers of Computer Science, 18*(6), Article 186345. https://doi.org/10.1007/s11704-024-40231-1

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629

Source code

The complete source code is submitted separately with this report as `BargainAgent-source.zip` (without API keys). `README.md` explains how to run it, and `docs/` contains the architecture, interfaces and evaluation details.

[If the code is published to GitHub before submission, add the repository link here.]